PRÁCTICA CALIFICADA DE LABORATORIO

Semanas 03 y 04: Arquitectura BDI y Aprendizaje por Refuerzo

Asignatura: Agentes Inteligentes (VIII Ciclo)	**Unidad:** I - Fundamentos y Arquitecturas Clásicas
Duración: 120 minutos	**Puntaje Total:** 20 Puntos (10 pts por ejercicio)

INSTRUCCIONES GENERALES

1. Lea detenidamente la especificación teórica y técnica de cada ejercicio antes de codificar.
2. Cada ejercicio debe ser desarrollado en archivos independientes de Python (practica_semana03_bdi.py y practica_semana04_rl.py).
3. Justifique las decisiones de diseño mediante comentarios en el código y asegúrese de que sus soluciones compilen y ejecuten sin errores sintácticos.

EJERCICIO 1 (SEMANA 03 — 10 PUNTOS): Agente Deliberativo BDI con Inferencia Deductiva y Reconsideración Dinámica

1. Contexto del Problema

Un **Agente SRE (Site Reliability Engineering)** autónomo supervisa la infraestructura de microservicios de una plataforma de comercio electrónico. El clúster opera bajo la arquitectura BDI (**Creencias, Deseos, Intenciones**) utilizando el intérprete de Wooldridge. El sistema enfrenta un escenario donde se combina saturación progresiva de memoria y ciberataques externos. El estudiante deberá programar las funciones internas de revisión de creencias (brf), generación de opciones (options), filtrado de intenciones (filter) y soporte de **planes alternativos (Fallback Plans)** ante fallos de cuota.

2. Especificación Técnica y Requerimientos

1. Inferencia Deductiva en brf (Revisión de Creencias): Extienda la función brf(B, p_t) para almacenar una ventana deslizante de las últimas 3 lecturas del porcentaje de CPU. Regla de Inferencia: Si las últimas 3 lecturas son estrictamente crecientes y la tasa de incremento supera +15% por ciclo, derive proactivamente la creencia 'anomalia_fuga_memoria = True' (incluso si la CPU aún no ha alcanzado el umbral crítico del 80%).

2. Deseos y Priorización en options y filter: Si 'anomalia_fuga_memoria == True', active el deseo PREVENIR_CAIDA_OOM con Prioridad 75. La jerarquía de prioridades debe quedar configurada así:

- MITIGAR_INTRUSION_SEGURIDAD (Prioridad 100)
- RESTAURAR_SERVICIO_CACHE (Prioridad 80)
- PREVENIR_CAIDA_OOM (Prioridad 75)
- ESTABILIZAR_RENDIMIENTO_SLA (Prioridad 60)

3. Manejo de Fallos de Plan y Fallback Plans: En la biblioteca de planes Π, registre dos planes para ESTABILIZAR_RENDIMIENTO_SLA:

• Plan Principal: Runbook_Autoescalado_Horizontal → ['APROVISIONAR_NODO_REPLICA_K8S', 'REBALANCEAR_TRAFICO_INGRESS'].

• Plan Secundario (Fallback): Runbook_Degradacion_Elegante_CircuitBreaker → ['ACTIVAR_CIRCUIT_BREAKER_RECOMENDACIONES', 'HABILITAR_RESPUESTA_ESTATICA_CDN'].

Si durante la ejecución la base de creencias detecta 'cuota_cloud_agotada == True', el agente debe marcar el plan principal como inviable, mantener la Intención y conmutar automáticamente al plan alternativo.

3. Plantilla de Código Python para Completar (practica_semana03_bdi.py)

```
from dataclasses import dataclass, field
from typing import List, Dict, Any, Optional, Callable

@dataclass
class TelemetriaCluster:
    ciclo: int
    cpu_pct: float
    latencia_ms: float
    alerta_siem: bool
    cuota_agotada: bool = False

@dataclass
class Deseo:
    nombre: str
    prioridad: int

@dataclass
class PlanBDI:
    nombre: str
    intencion_meta: str
    precondicion: Callable[[Dict[str, Any]], bool]
    cuerpo_acciones: List[str]

class AgenteBDI_Estudiante:
    def __init__(self):
        self.creencias: Dict[str, Any] = {
            "historial_cpu": [],
            "anomalia_fuga_memoria": False,
            "bajo_ataque": False,
            "cuota_cloud_agotada": False,
            "plan_fallido": None
        }
        self.intencion_actual: Optional[Deseo] = None
        self.plan_activo: Optional[PlanBDI] = None
        self.cola_acciones: List[str] = []
```

```
    def brf(self, p: TelemetriaCluster) -> None:
        """TAREA 1: Actualizar creencias e implementar la regla deductiva de fuga de
memoria."""
        self.creencias["historial_cpu"].append(p.cpu_pct)
        if len(self.creencias["historial_cpu"]) > 3:
            self.creencias["historial_cpu"].pop(0)

        # TODO: Implementar regla de inferencia sobre ventana de 3 lecturas
        # Si p1 < p2 < p3 y (p3 - p1) >= 15.0 -> setear anomalia_fuga_memoria = True
        pass

    def options(self) -> Dict[str, Deseo]:
        """TAREA 2: Generar conjunto de deseos D según creencias."""
        opciones = {}
        # TODO: Generar deseos según prioridades (100, 80, 75, 60)
        return opciones

    def filter(self, opciones: Dict[str, Deseo]) -> Optional[Deseo]:
        """TAREA 3: Seleccionar la intención con compromiso de mente abierta."""
        if not opciones:
            return None
        return max(opciones.values(), key=lambda d: d.prioridad)

    def seleccionar_plan_fallback(self, intencion: Deseo, biblioteca: List[PlanBDI]) -
> Optional[PlanBDI]:
        """TAREA 4: Seleccionar plan excluyendo planes marcados como fallidos."""
        # TODO: Retornar plan aplicable cuya precondición se cumpla y no sea el
plan_fallido
        return None

if __name__ == "__main__":
    print("=== EVALUACIÓN EJERCICIO 1: AGENTE BDI ===")
    agente = AgenteBDI_Estudiante()
    # Ejecutar simulación con lecturas de CPU [40.0, 58.0, 75.0] (Inferencia de fuga
de memoria)
```

EJERCICIO 2 (SEMANA 04 — 10 PUNTOS): Aprendizaje por Refuerzo con Q-Learning en Entorno Estocástico y Reward Shaping

1. Contexto del Problema

Un dron de reparto logístico opera en un entorno estocástico modelado como un **Proceso de Decisión de Markov (MDP)** ⟨S, A, P, R, γ⟩. Debido a ráfagas de viento costero, el entorno tiene una estocasticidad del 33.3% de éxito en la dirección deseada (similar al entorno FrozenLake-v1 con is_slippery=True). El

estudiante debe implementar el algoritmo **Q-Learning** (Model-Free) utilizando la actualización por Error de Diferencia Temporal (TD-Error) y evaluar el impacto de la **Ingeniería de Recompensas (Reward Shaping)** en la velocidad de convergencia de la política óptima $\pi^*(s)$.

2. Especificación Técnica y Requerimientos

1. Ecuación de Actualización TD-Error (Q-Learning): Implemente en código estricto la regla de actualización de Bellman sin modelo:

- TD-Target = r + γ · max_a' Q(s', a')
- TD-Error = TD-Target - Q(s, a)
- Q(s, a) ← Q(s, a) + α · TD-Error

2. Políticas de Exploración ε-Greedy: Implemente el esquema de exploración vs. explotación con decaimiento exponencial de ε desde $\varepsilon_0 = 1.0$ hasta ε_min = 0.01 con factor 0.995.

3. Experimento de Reward Shaping (Ingeniería de Recompensas): • Entorno Estándar: Recompensa de +1.0 únicamente al alcanzar la meta (G), y 0.0 en celdas normales (F) o caída en hoyo (H).
• Entorno Modificado (Reward Shaping): Penalización por paso r_paso = -0.01 (estimular la brevedad de ruta), penalización severa por caer en agujero r_agujero = -1.0, y recompensa de meta r_meta = +1.0.
Muestre mediante reporte de consola la tasa de éxito media sobre bloques de episodios.

3. Plantilla de Código Python para Completar (practica_semana04_rl.py)

```
import numpy as np
import random
from typing import Tuple

class EntornoGridworldEstocastico:
    """Entorno Gridworld 4x4 Estocástico (Simulación reducida de FrozenLake)."""
    def __init__(self, reward_shaping: bool = False):
        self.filas = 4
        self.columnas = 4
        self.n_estados = 16
        self.n_acciones = 4  # 0: Izquierda, 1: Abajo, 2: Derecha, 3: Arriba
        self.agujeros = {5, 7, 11, 12}
        self.meta = 15
        self.estado_actual = 0
        self.reward_shaping = reward_shaping

    def reset(self) -> int:
        self.estado_actual = 0
        return self.estado_actual

    def step(self, accion: int) -> Tuple[int, float, bool]:
        # TAREA 1: Simular estocasticidad (33% acción deseada, 66% desvío lateral)
        if random.uniform(0, 1) < 0.66:
            accion = (accion + random.choice([-1, 1])) % 4
```

```
        # Transición determinista tras aplicar el viento
        r, c = divmod(self.estado_actual, 4)
        if accion == 0: c = max(0, c - 1)
        elif accion == 1: r = min(3, r + 1)
        elif accion == 2: c = min(3, c + 1)
        elif accion == 3: r = max(0, r - 1)

        nuevo_estado = r * 4 + c
        self.estado_actual = nuevo_estado

        # TAREA 2: Asignar recompensas según el esquema activado
        terminado = False
        if nuevo_estado in self.agujeros:
            terminado = True
            recompensa = -1.0 if self.reward_shaping else 0.0
        elif nuevo_estado == self.meta:
            terminado = True
            recompensa = 1.0
        else:
            recompensa = -0.01 if self.reward_shaping else 0.0

        return nuevo_estado, recompensa, terminado

def entrenar_q_learning(env, episodios=2000, alpha=0.8, gamma=0.95):
    q_table = np.zeros((env.n_estados, env.n_acciones))
    epsilon = 1.0

    for ep in range(episodios):
        state = env.reset()
        done = False

        while not done:
            # TODO: Implementar selección de acción Epsilon-Greedy
            if random.uniform(0, 1) < epsilon:
                action = random.randint(0, env.n_acciones - 1)
            else:
                action = int(np.argmax(q_table[state, :]))

            next_state, reward, done = env.step(action)

            # TODO: Implementar actualización TD-Error de Q-Learning
            td_target = reward + gamma * np.max(q_table[next_state, :])
            td_error = td_target - q_table[state, action]
            q_table[state, action] += alpha * td_error

            state = next_state
```

```
        epsilon = max(0.01, epsilon * 0.995)

    return q_table

if __name__ == "__main__":
    print("=== EVALUACIÓN EJERCICIO 2: Q-LEARNING Y REWARD SHAPING ===")
    env_estandar = EntornoGridworldEstocastico(reward_shaping=False)
    q_estandar = entrenar_q_learning(env_estandar)

    env_shaping = EntornoGridworldEstocastico(reward_shaping=True)
    q_shaping = entrenar_q_learning(env_shaping)

    print("\nTabla Q Entorno Estándar (Estado 0):", np.round(q_estandar[0, :], 3))
    print("Tabla Q Reward Shaping (Estado 0):", np.round(q_shaping[0, :], 3))
```

RÚBRICA DE EVALUACIÓN CUALITATIVA Y CUANTITATIVA (20 PUNTOS)

Criterio	Sobresaliente	En Proceso	Insuficiente
Ejercicio 1: Inferencia Deductiva en brf (3 pts)	Modela la ventana de 3 lecturas de CPU, deduce anomalia_fuga_memoria correctamente y activa alerta previa al 80%. (3.0 pts)	Procesa el historial pero requiere llegar al 80% para activar la alerta. (1.5 pts)	No implementa la inferencia en brf. (0 pts)
Ejercicio 1: Priorización y Fallback Plans (7 pts)	Gestiona prioridades BDI y conmuta automáticamente al plan de Circuit Breaker preservando la Intención ante falta de cuota. (7.0 pts)	Conmuta de plan pero pierde la Intención o re-intenta infinitamente el plan fallido. (3.5 pts)	No implementa el mecanismo de Fallback. (0 pts)
Ejercicio 2: Algoritmo Q-Learning & TD-Error (5 pts)	Implementa correctamente la ecuación de Bellman model-free y la estrategia ε-greedy con decaimiento. (5.0 pts)	Aplica la fórmula pero con errores en el cálculo del objetivo TD o sin decaimiento de ε. (2.5 pts)	No calcula el TD-Error. (0 pts)
Ejercicio 2: Análisis de Reward Shaping (5 pts)	Demuestra experimentalmente cómo la penalización por paso y por hoyo acelera la convergencia y evita trayectorias en bucle. (5.0 pts)	Aplica las recompensas pero la penalización destruye la convergencia hacia la meta. (2.5 pts)	No modifica la función de recompensa. (0 pts)